

毕设论文 — 输入模板 v9

工作流：分 3 轮使用，每轮指定 Part 编号 + 填写该 Part 核心信息 → Claude 输出对应章节（md 格式，正文内容）。

目标字数：≥ 1.6 万字（**无上限**，按任务量自由扩展）

核心原则：你只填项目核心思路/方案/素材相关的内容，其余（论文常规话术、过渡段落、通用描述）Claude 自由发挥。

⚠ 全局硬约束（Claude 严格遵守）

- 所有输入图片必须出现在最终论文中：**你提供的每一张图片，Claude 必须插入论文相应章节，并对图片内容做实质性展开（不少于 100 字/张）。一张都不许丢（除了G1. 项目背景与任务需求书中的图片）。
- 所有输入代码必须有对应章节解释：**你贴的每段应用层代码，Claude 必须为其撰写对应小节，逐函数解释功能/输入输出/实现思路/调用关系。代码不能"贴而不讲"。
- 目录由 Claude 根据任务量动态决定：**模板 G2 中的目录仅作大致参考，Claude 可根据项目实际复杂度增减小节、调整名称，使论文逻辑通顺。
- 字数下限 16000，最多21000字：**第二章、第三章、第四章(下位机软件部分)是论文主体，尽可能详细；其他章节按需扩展。
- 摘要严格 ≤ 300 字**（中文摘要）。
- 第六章测试必须结合项目背景深入解释：**不能只是"看图说话"，要把测试现象与设计目标、技术原理、实际应用场景联系起来分析。

图片命名规则

图片放在 md 文件同级目录下，按以下命名约定，Claude 在论文中按顺序自动引用：

- 任务书：任务书.png
- 系统总体流程图：系统流程图.png
- 系统结构图：系统结构图.png
- 系统电源树：电源树.png
- 模块图（用于第二章器件选型）：模块_1.png、模块_2.png
- 原理图（用于第三章硬件设计）：原理图_1.png、原理图_2.png
- 软件流程图（用于第四章软件设计）：软件流程图_1.png、软件流程图_2.png
- 机械章节：机械_1.png、机械_2.png
- 测试章节：测试_1.png、测试_2.png

全局信息（Part 1 一次性填写，三轮复用）

G1. 项目背景与任务需求书

粘贴项目背景描述和任务需求书全文，或贴任务书图片。

Claude 会从中提取信息用于摘要、绪论、设计目标、创新点、关键词等部分。

项目名称：

任务书：

设计以单片机为核心的最小系统电路、超声波传感器驱动与回波检测电路、LCD 显示电路、按键输入电路及声光报警电路等；同时，使用 C 语言等编程工具开发相应的主控程序、超声波发射/接收时序控制程序、距离计算与液位转换算法、显示程序及报警判断程序，完成从硬件设计制作到软件编程调试的全过程。↵

↵

具体功能及指标要求如下：↵

(1) 具备非接触式液位测量功能，系统能通过超声波模块周期性地向液面发射超声波并接收回波，准确测量出传感器到液面的距离。↵

(2) 系统测量范围应覆盖常见中小型容器，设定为 5 cm 至 100 cm。在整个量程内，静态液位的测量误差不大于 ± 3 mm。↵

(3) 具备液位变化量计算与显示功能，系统能通过记录初始液位高度，并计算与当前测量的差值，实时显示液位的高度变化量，变化量显示分辨率达到 1 mm。↵

(4) 具备高低液位报警功能，用户可通过按键自行设定液位上限和下限报警阈值。当液位超过设定范围时，系统能立即通过蜂鸣器和 LED 指示灯发出声光报警信号，报警响应延迟 ≤ 1 秒。↵

(5) 使用 LCD 液晶显示屏清晰、直观地显示信息，包括：实时测量距离值、计算得出的当前液位高度、液位变化量、设定的报警阈值以及系统状态（正常/报警）。↵

(6) 系统应具备良好的稳定性和抗干扰能力。在室温环境下，对静止液面进行连续测量，其读数波动范围不超过 ± 3 mm。系统应能有效处理因液面轻微波动或环境中突发噪音导致的偶然误测。↵

【或在此粘贴文字版任务需求书】

G2. 论文目录（参考）

以下目录仅作参考。Claude 可根据项目实际任务量增减小节、调整名称，使论文结构与项目复杂度匹配。

摘要

Abstract

第一章 绪论

1.1 课题的研究背景与意义

1.1.1 (项目名称)国内外研究现状

1.1.2 (项目名称)的价值与意义

第二章 系统方案设计

2.1 需求分析

2.2 系统设计方案对比

2.3 系统总体设计方案（含器件选型与开发工具介绍）

```
# 第三章 硬件设计
## 3.1 原理图1设计（根据输入的原理图图片和备注自拟小节名称）
## 3.2 原理图2设计
## 3.x 原理图x设计...
# 第四章 软件设计
## 4.1 单片机软件设计
### 4.1.1 FreeRTOS 操作系统（本论文不写该小节）
### 4.1.2 STM32 基础外设配置（本论文不写该小节）
### 4.1.3 功能 1 软件设计（根据应用层代码和功能流程图自拟小节名称）
### 4.1.4 功能 2 软件设计
### 4.1.x 功能 x 软件设计...
## 4.2 上位机软件设计
### 4.2.1 上位机与下位机通信方案
### 4.2.2 前端用户 UI 设计
### 4.2.3 后端操作逻辑设计
# 第五章 机械外观设计
## 5.1 零件 1 设计
## 5.x 零件 x 设计...
# 第六章 系统测试
## 6.1 功能 1 测试
## 6.x 功能 x 测试...
# 第七章 总结与展望
```

G3. 输出格式要求

- 输出语言：中文（摘要附英文版）
- 文件格式：Markdown
- 章节编号：以 G2 目录为参考，Claude 可调整
- 图表编号：图 X-Y / 表 X-Y（章号-序号）
- 公式编号：(X-Y)
- 图片占位：用 `![图 X-Y 说明](.\图片名.png)` 引用，按本模板顶部命名规则
- 流程图/框图（Claude 补绘的）：用 Mermaid 代码块
- 字数：≥ 16000 字，无上限

G4. 额外备注（可选）

默认要求，可修改。这些规则在三轮输出中均严格执行。

1. 输出正文一定要口语化，尽可能多使用语气词或第一人称动词。比如：了，啊，吧，我，笔者，本人。每一千字最多出现10次语气词，不要刻意去加语气词，语言不要过于牵强，平静的描述。
2. 正文中不要用小括号去解释内容，把要用小括号说明的内容直接写进正文里，用逗号隔开。
3. 文本中只使用逗号，句号，省略号，顿号。不要使用其它任何标点符号。
4. 需要有错别字，每 1000 字至少出现 5 处错别字。
5. 需要有语句表达不通顺的句子，每 5000 字至少有 5 处表达不通顺的语句。
6. 禁止在论文中出现“分层架构”的概念。也不需要说明该项目是用分层架构完成的。

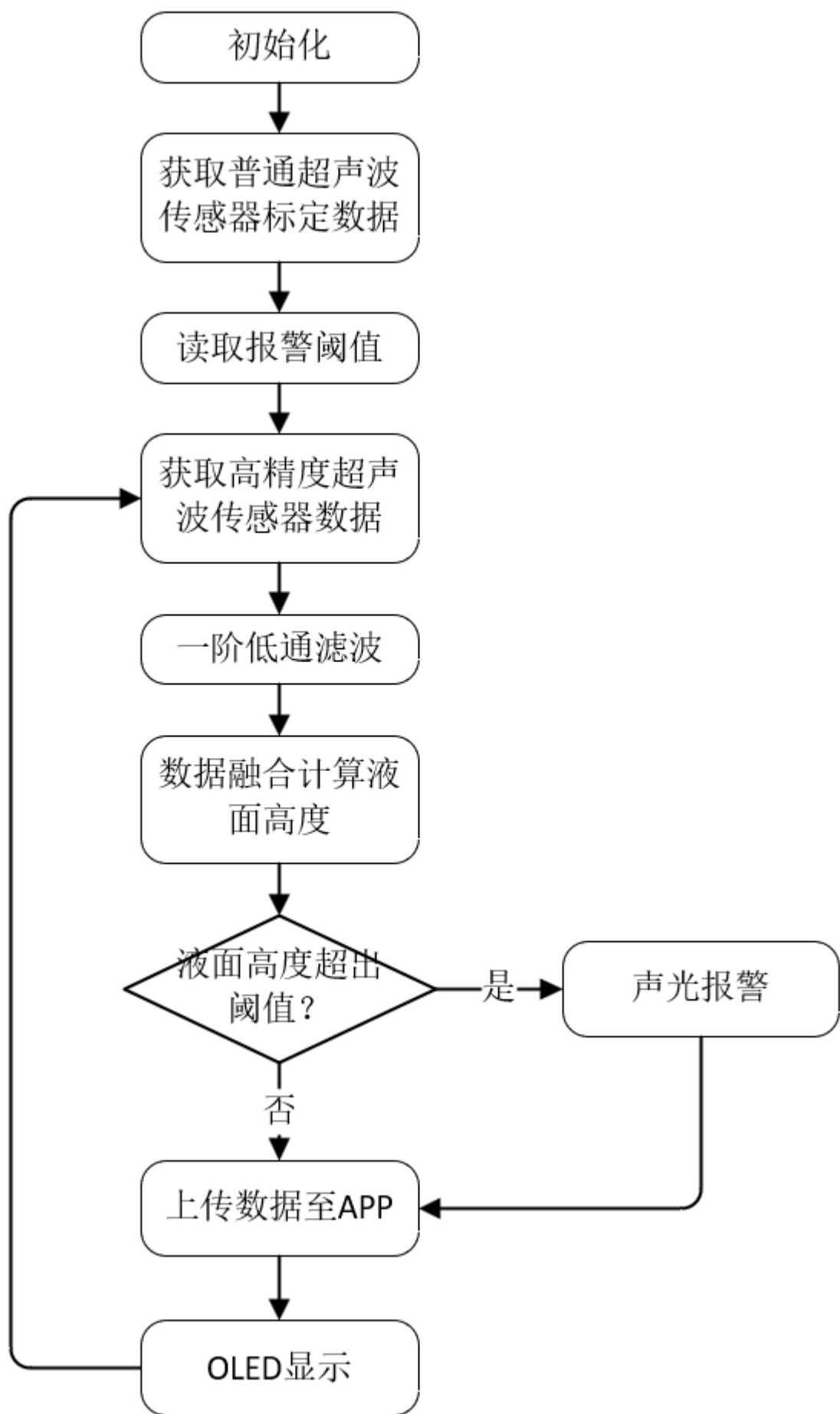
【可在下方追加其他要求】

G5. 系统框图

第二章 2.3 节会引用以下三张图来描述整机架构。每张图都必须出现在论文中并展开描述。

G5.1 系统总体流程图

描述系统从开机到主循环的工作流程，体现各功能模块在时间维度上的执行关系。

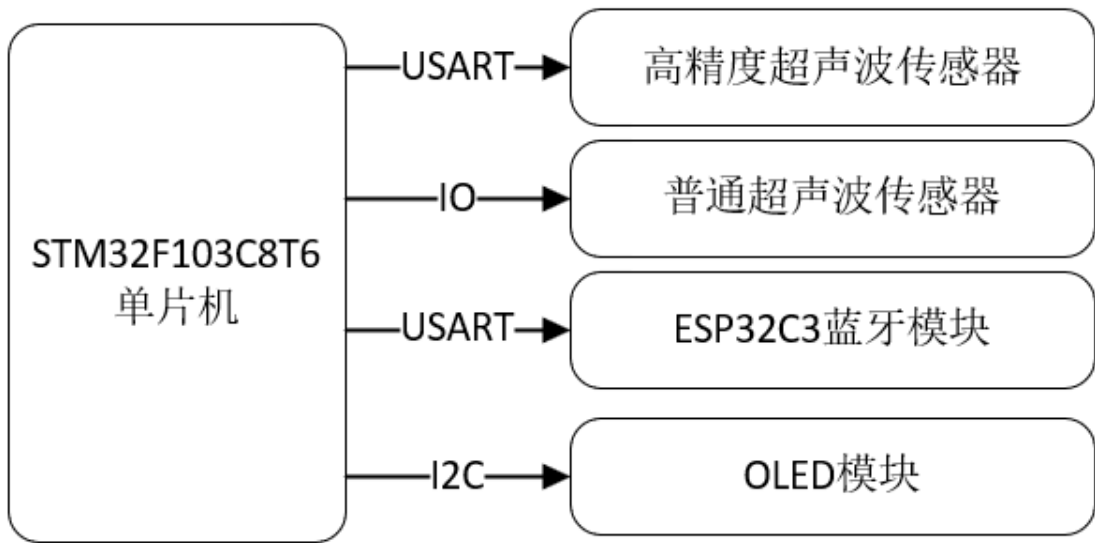


描述（可选）：

系统有两个超声波传感器构成，安装方式是一前一后。前面的超声波传感器是高精度传感器，朝向液面。后面的超声波传感器是普通传感器，面向地面。系统上电先采集10次普通超声波传感器数据计算均值获取距地面高度，再周期采集高精度超声波传感器获取当前实时距液面高度。则液位高度=距地面高度-距液面高度。用液位高度和阈值判断是否需要报警。

G5.2 系统结构图

描述各硬件模块、软件模块之间的连接关系和数据流，体现系统在空间维度上的组织。

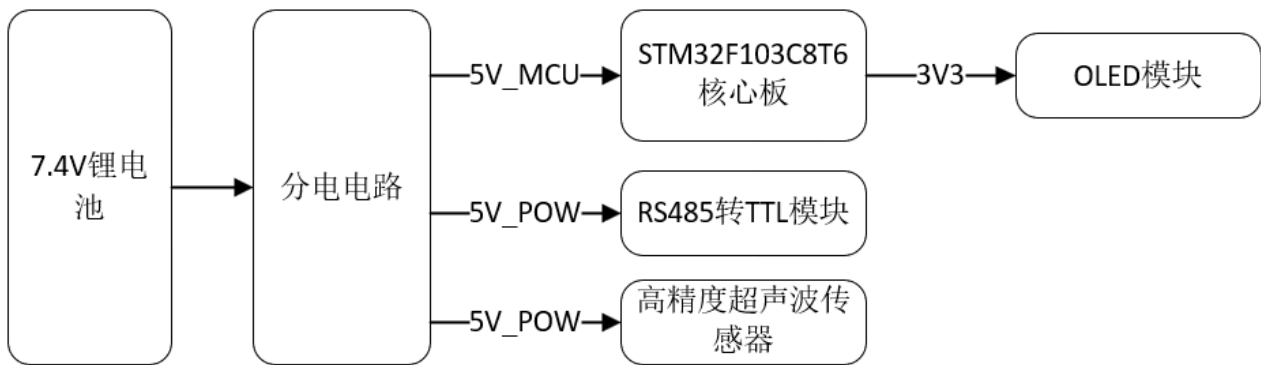


描述（可选）：

G5.3 系统电源树

描述电源从源头到各模块的分配关系。

本项目电源方案为固定方案：7.4V 锂电池供电 → 两路 LDO 稳压输出 5V + 一路 DCDC 升压 12V（若有）。Claude 会根据电源树图自动判断本项目用到了哪些电源支路。



描述（可选）：

可以说明高精度超声波传感器的功率较大,使用单独一路5V

G6. 开发工具（按需介绍）

第二章 2.3 节会基于此表对**实际用到的**开发工具做简短介绍。
没用到的工具直接删除该行，Claude 不会介绍。

名称	版本	功能	备注
Altium Designer	2025	硬件原理图与 PCB 开发	/
KEIL	MDK-ARM V5.41	STM32 下位机软件开发	编译器使用 Compiler Version 6
SOLIDWORKS	2025	机械结构开发	/
STM32CubeMX	6.16.0	STM32 底层软件配置	/
Android Studio	2025.2.3.9	上位机 APP 开发	/

Claude 介绍工具的判断规则：

- KEIL / STM32CubeMX：必介绍（任何 STM32 项目都会用到）
- Altium Designer：第三章有原理图则介绍
- SOLIDWORKS：第五章有机械结构则介绍
- Android Studio：项目含上位机 APP 则介绍

Part 1：摘要 + 绪论 + 系统方案设计（含器件选型）

触发指令： 输出 Part 1
输出章节： 摘要（≤200 字） / Abstract / 第一章绪论 / 第二章系统方案设计
预计字数： 5000~6000 字（第二章必须详细，可继续扩展）
依赖： G1~G6 + 本 Part 输入

P1.1 国内外研究现状素材（可选但强烈建议）

列举 2~4 个相关方案/产品/论文，Claude 据此撰写国内外现状段落。
不填则 Claude 按行业常识生成通用描述，真实性较弱。

格式随意，例：

- 国外：某产品/论文，简述特点和不足
- 国内：某产品/论文，简述特点和不足

【在此填写】

P1.2 方案对比（可选）

如果前期做过方案对比，简述被否方案和最终方案。
不填则 Claude 按"通用方案 vs 本设计"格式编 1~2 个对比方案。
【在此填写，留空则 Claude 自由发挥】

P1.3 器件选型素材（用于 2.3 系统总体设计方案）

提供本项目所用模块的图片 + 简短描述。
Claude 在 2.3 节对**每一个模块**做介绍，包括：型号、关键参数、工作原理、在系统中的作用，以及为什么选用。
图片按 模块_1.png、模块_2.png 顺序命名。每张图必须出现在论文中。

模块 1:



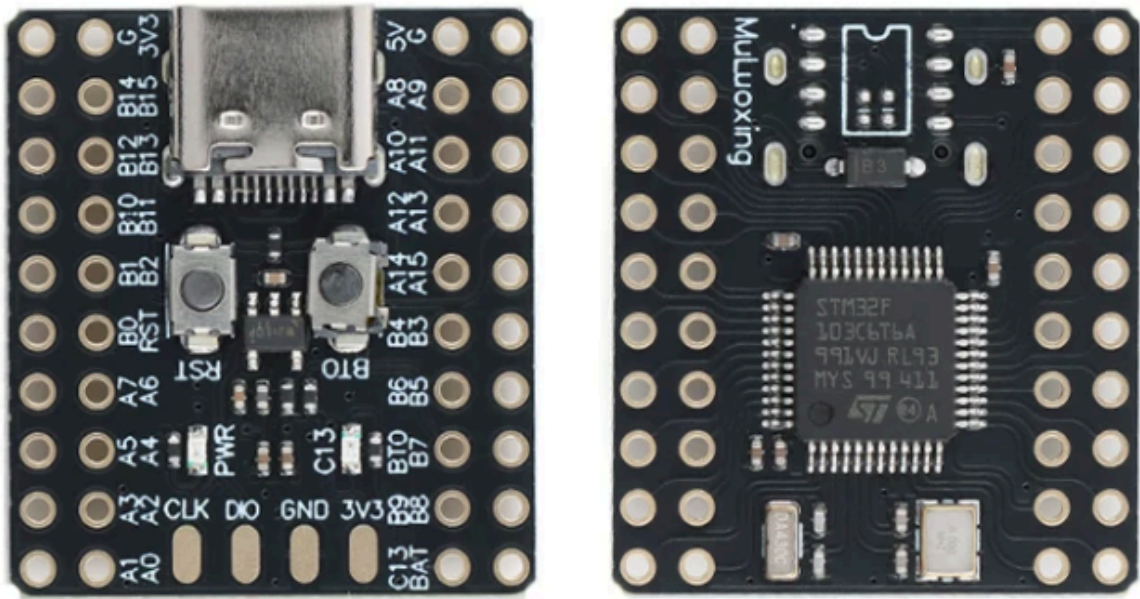
- 描述：工业级TTL转RS485通讯模块双隔离设计高速传输抗干扰电平转换模块

模块 2:



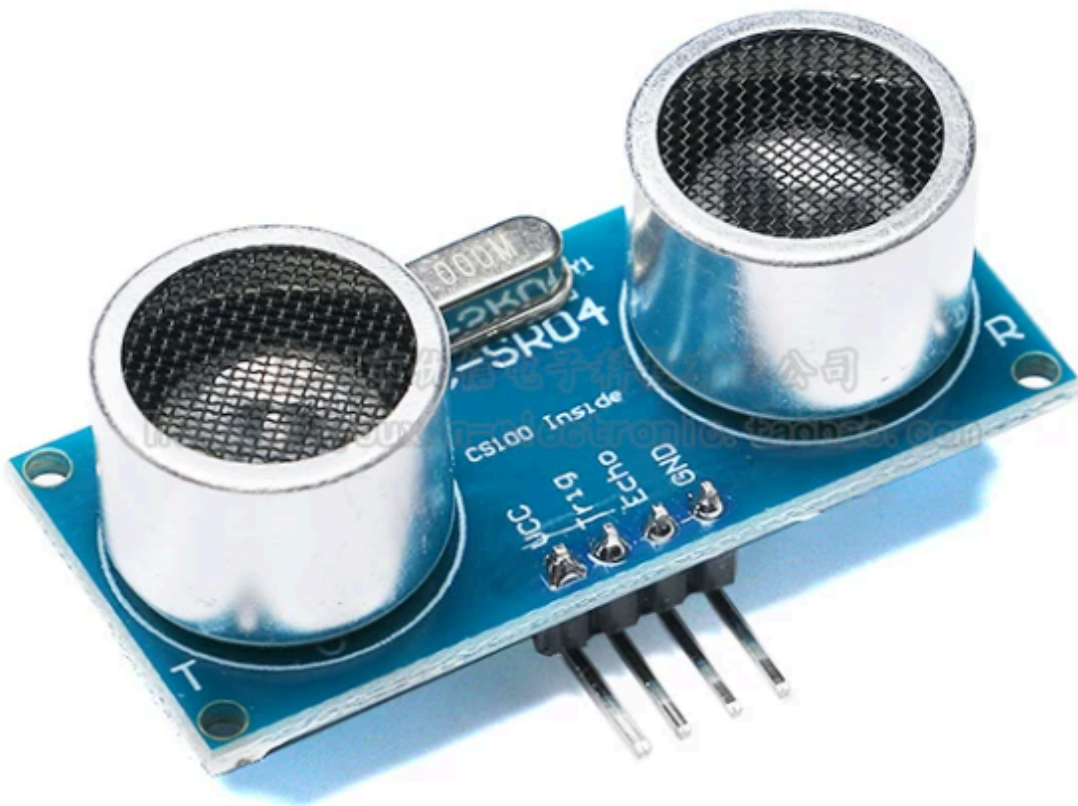
- 描述：超声波雷达测距传感器超声波传感器人体测量机器人避障距离感应器,HL-ULT1-4M RS485 Modbus-RTU.modelbus协议,分辨率1mm

模块 3:



- 描述: STM32F103C8T6_mini

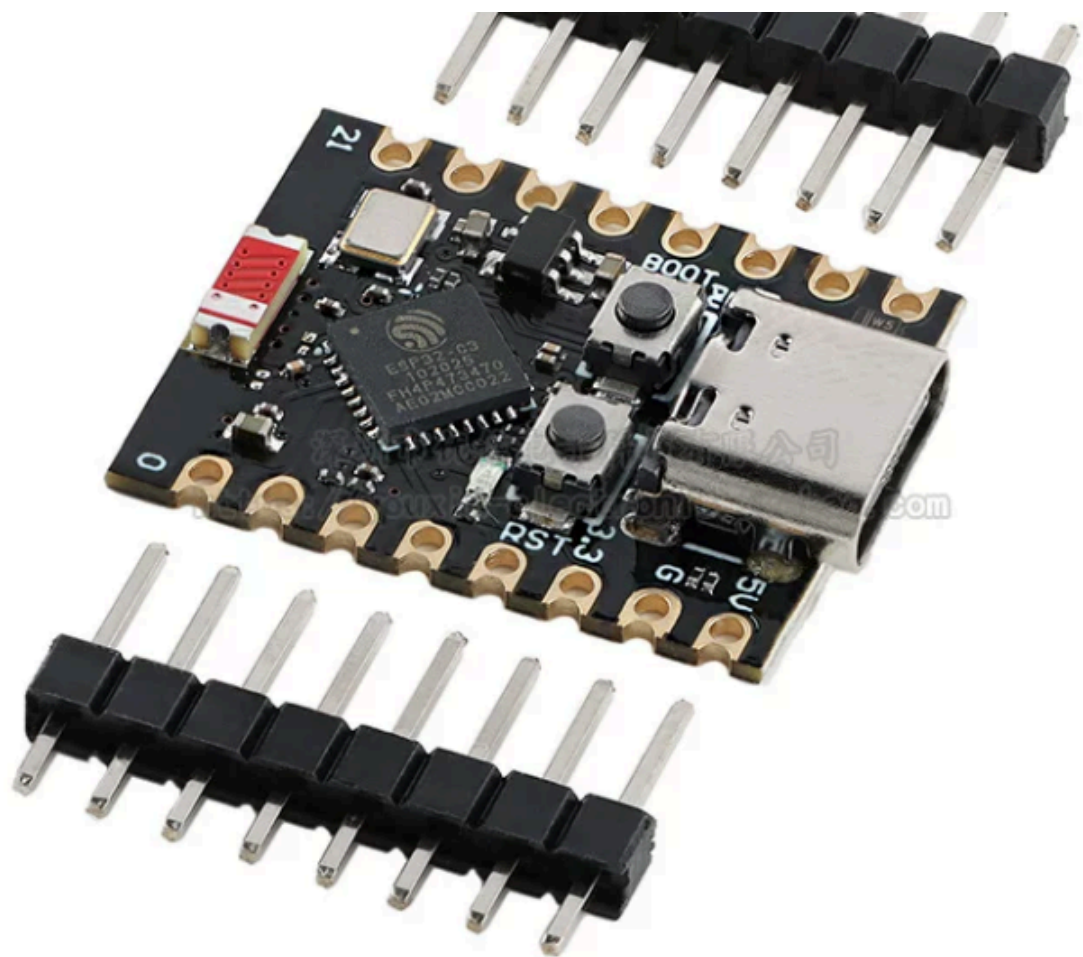
模块 4:



- 描述: HC-SR04 新版 超声波测距模块 宽电压3-5.5V 工业级 传感器

(按需添加更多)

模块 5:



- 描述: ESP32-C3-PRO/SuperMini开发板 Wi-Fi蓝牙编程学习控制器模块

(按需添加更多)

Part 2: 硬件设计 + 软件设计 + 机械外观设计

触发指令: 输出 Part 2

输出章节: 第三章硬件设计 / 第四章软件设计 / 第五章机械外观设计 (若有)

预计字数: 8000~10000 字 (第三、四章是论文主体, 必须详细)

依赖: G1~G6 + 本 Part 输入

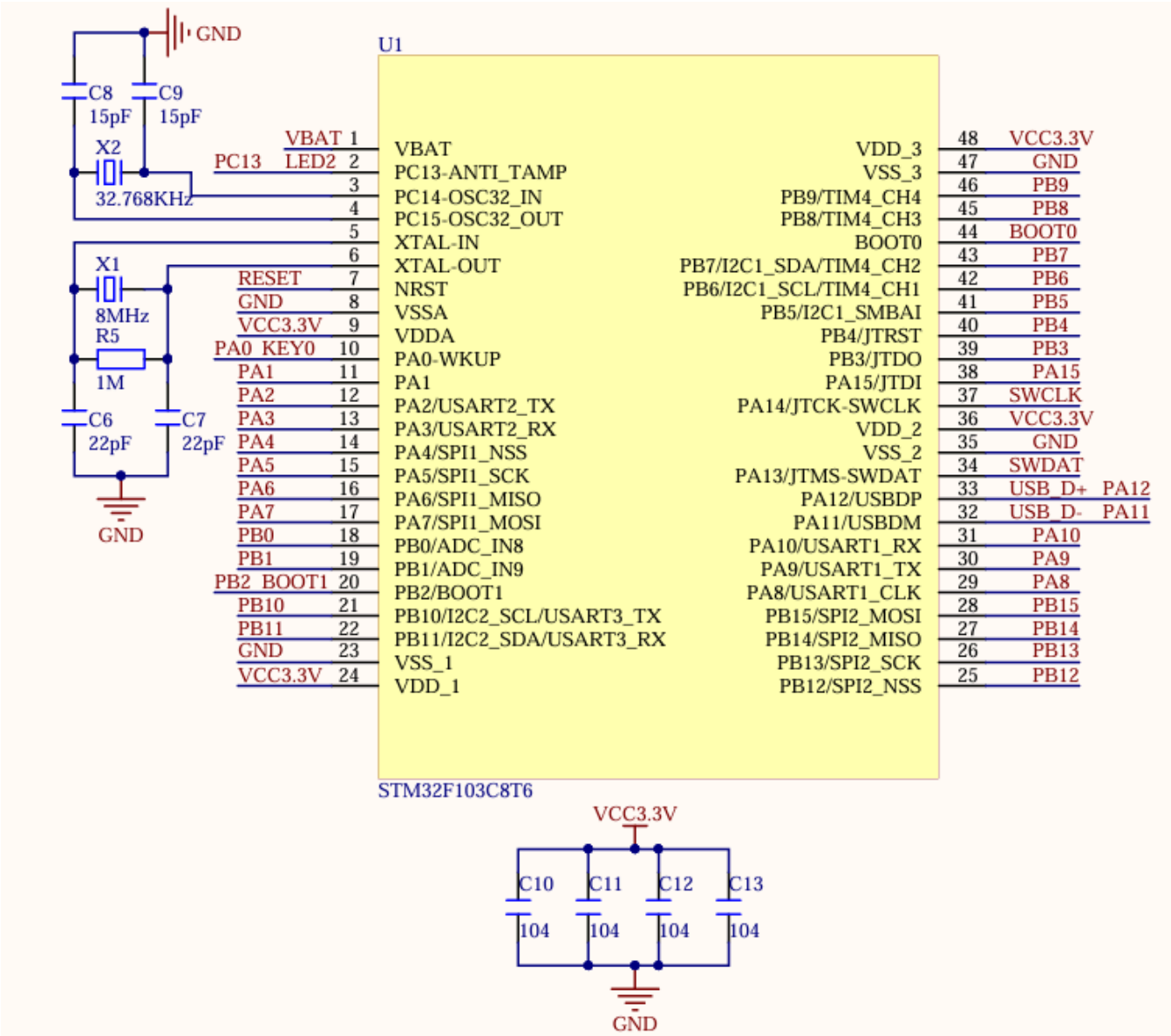
P2.1 原理图素材 (用于第三章)

第三章按原理图数量拆分小节, 每张原理图对应一个小节, 小节名由 Claude 根据原理图内容和你的描述自拟。Claude 会针对每张原理图描述其电路构成、关键器件、信号走向、设计要点、参数计算 (如分压、滤波、上下拉等)。

每张图必须出现在论文中并详细展开。

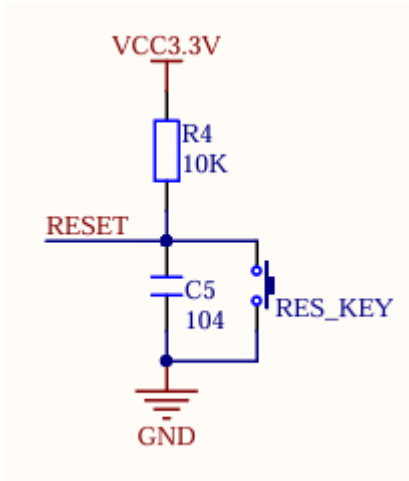
图片按 原理图_1.png、原理图_2.png 顺序命名。

原理图 1:



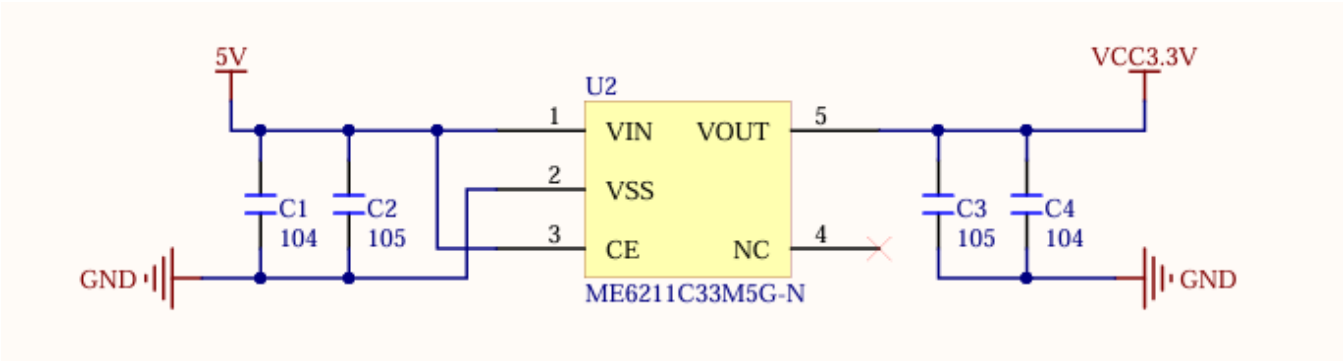
- 描述：STM32最小系统_芯片部分

原理图 4：



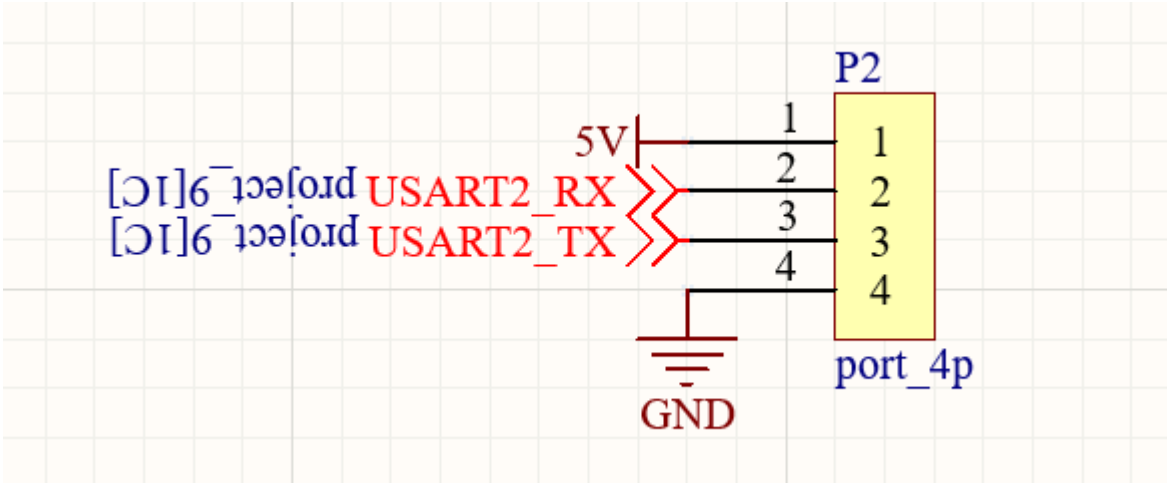
- 描述：STM32最小系统_复位部分

原理图 5：



- 描述: STM32最小系统_板载LDO

原理图 6:



- 描述: RS485模块接口,超声波模块用的RS485协议,要板载RS485转TTL

(按需添加更多)

P2.2 软件素材（用于第四章）

P2.2.1 应用层代码

粘贴 task_user1.c / task_user1.h 全部代码，或挑核心应用层代码。

重要：选取核心的函数Claude 都会为其撰写对应小节，选取核心的函数讲解。

driver_sensor.h:

```
#include "driver_sensor.h"
#include <string.h>

/*=====
 * 私有宏定义
 *=====*/

/* Modbus RTU 请求帧长度（字节）。 */
#define ULT_REQ_LEN      (8U)
```

```

/* Modbus RTU 应答帧长度（字节）。 */
#define ULT_RSP_LEN      (7U)
/* 应答帧功能码字节索引。 */
#define ULT_RSP_FC_IDX   (1U)
/* 应答帧字节数字节索引。 */
#define ULT_RSP_BC_IDX   (2U)
/* 应答帧距离高字节索引。 */
#define ULT_RSP_DH_IDX   (3U)
/* 应答帧距离低字节索引。 */
#define ULT_RSP_DL_IDX   (4U)
/* 应答帧 CRC 低字节索引。 */
#define ULT_RSP_CRCL_IDX (5U)
/* 应答帧 CRC 高字节索引。 */
#define ULT_RSP_CRCH_IDX (6U)

/*=====
 * 私有类型定义
 *=====*/

/* 应答帧接收状态（由字节回调填充，主任务轮询）。 */
typedef struct {
    uint8_t buf[ULT_RSP_LEN]; /* 应答帧字节缓存 */
    uint8_t cnt;               /* 已接收字节数 */
    uint8_t ready;             /* 帧接收完成标志（1: 完成；0: 未完成） */
} ultRspState_t;

/*=====
 * 私有变量
 *=====*/

/* 超声波传感器信息（公有，供上层直接访问）。 */
senserInfo_t senserInfo;

/* 应答帧接收状态（私有）。 */
static ultRspState_t ultRspState;

/* Modbus RTU 读距离请求帧（设备地址 0x01，功能码 0x03，寄存器 0x0000，数量 1）。
 * CRC16: 0x840A（低字节 0x84 在前）。
 */
static uint8_t ultReqFrame[ULT_REQ_LEN] = {
    0x01U, 0x03U, 0x00U, 0x00U, 0x00U, 0x01U, 0x84U, 0x0AU
};

/*=====
 * 私有函数
 *=====*/

/* 计算 Modbus RTU CRC16（多项式 0xA001，初始值 0xFFFF）。
 * data : 待校验字节数组首地址；
 * len  : 字节数。
 */
static uint16_t __DRIVER_SENSEN_CRC16Calc(const uint8_t *data, uint8_t len){
    uint16_t crc = 0xFFFFU;

```

```

uint8_t i, j;
for(i = 0U; i < len; i++){
    crc ^= (uint16_t)data[i];
    for(j = 0U; j < 8U; j++){
        if(crc & 0x0001U){
            crc = (crc >> 1U) ^ 0xA001U;
        } else {
            crc >>= 1U;
        }
    }
}
return crc;
}

/* 自定义字节回调：由 stdlib_usart 在中断中逐字节触发，收集 Modbus RTU 应答帧。
 * 收满 ULT_RSP_LEN 字节后置位 ready 标志，多余字节自动丢弃。
 */
static void __DRIVER_SENSEN_ULTParseCb(uint8_t port, uint8_t byte){
    (void)port;
    if(ultrspState.cnt < ULT_RSP_LEN){
        ultrspState.buf[ultrspState.cnt] = byte;
        ultrspState.cnt++;
        if(ultrspState.cnt >= ULT_RSP_LEN){
            ultrspState.ready = 1U;
        }
    }
}

/*=====
 * API接口
 *=====*/

/* AHT10 初始化命令数据字节（0xE1 命令后跟 0x08 0x00）。 */
static const uint8_t aht10InitData[2] = {0x08U, 0x00U};
/* AHT10 触发采集命令数据字节（0xAC 命令后跟 0x33 0x00）。 */
static const uint8_t aht10TrigData[2] = {0x33U, 0x00U};

/* 初始化传感器驱动：
 * 1. 注册 UART_PORT2 字节回调以接收 Modbus RTU 应答帧（波特率由 CubeMX 配置为 9600）；
 * 2. 将 TRIG 引脚置低，准备 HC-SR04 触发；
 * 3. 向 AHT10 发送初始化命令并等待校准完成。
 */
void DRIVER_SENSEN_Init(void){
    memset(&senserInfo, 0, sizeof(senserInfo));
    memset(&ultrspState, 0, sizeof(ultrspState));
    SENSER_ULT_DEP_UART_SET_CB(__DRIVER_SENSEN_ULTParseCb);
    SENSER_HCSR04_DEP_TRIG_SET(GPIO_LEVEL_LOW);
    /* AHT10 初始化：发送 0xE1 命令，等待 75ms 完成校准 */
    SENSER_AHT10_DEP_WRITE(0xE1U, aht10InitData, 2U);
    SENSER_AHT10_DEP_DELAY_MS(SENSER_AHT10_INIT_WAIT_MS);
}

/* 发送 Modbus RTU 读距离请求，阻塞等待应答（最长 SENSER_ULT_TIMEOUT_MS ms）。

```

```

* 距离计算公式:  $\text{distMm} = \text{应答高字节} \times 256 + \text{应答低字节}$  (单位 mm)。
* distMm : 输出参数, NULL 时仅更新 senserInfo, 不写出;
* 返回值 : 成功 (CRC 通过且帧格式正确) 返回 1, 超时或校验失败返回 0。
*/
uint8_t DRIVER_SENSE_GetDistance(uint16_t *distMm){
    uint16_t crc, rspCrc;
    uint32_t startMs;

    /* 清除上次应答状态, 准备接收新帧 */
    ulRspState.cnt = 0U;
    ulRspState.ready = 0U;

    /* 发送 Modbus RTU 请求帧 */
    SENSER_ULT_DEP_UART_SEND_RAW(ulReqFrame, ULT_REQ_LEN);

    /* 阻塞等待应答帧接收完成 */
    startMs = HAL_GetTick();
    while(ulRspState.ready == 0U){
        if((HAL_GetTick() - startMs) >= SENSER_ULT_TIMEOUT_MS){
            return 0U;
        }
    }

    /* 校验 CRC16 (覆盖应答帧前 5 字节, 即不含 CRC 的部分) */
    crc = __DRIVER_SENSE_Crc16Calc(ulRspState.buf, ULT_RSP_LEN - 2U);
    rspCrc = (uint16_t)ulRspState.buf[ULT_RSP_CRCL_IDX] |
        ((uint16_t)ulRspState.buf[ULT_RSP_CRCH_IDX] << 8U);
    if(crc != rspCrc){
        return 0U;
    }

    /* 校验设备地址、功能码与字节数 */
    if(ulRspState.buf[0] != SENSER_ULT_DEVICE_ADDR ||
        ulRspState.buf[ULT_RSP_FC_IDX] != 0x03U ||
        ulRspState.buf[ULT_RSP_BC_IDX] != 0x02U){
        return 0U;
    }

    /* 解析距离值并更新 senserInfo */
    senserInfo.distMm = ((uint16_t)ulRspState.buf[ULT_RSP_DH_IDX] << 8U) |
        (uint16_t)ulRspState.buf[ULT_RSP_DL_IDX];
    senserInfo.isValid = 1U;

    if(distMm != NULL){
        *distMm = senserInfo.distMm;
    }
    return 1U;
}

/* HC-SR04: 发送 10us 触发脉冲, 阻塞测量回波时间并计算距离。
* 时序: TRIG 低 2us → TRIG 高 10us → TRIG 低 → 等待 ECHO 高 → 计时 → 等待 ECHO 低 → 计算距离。
* 距离公式:  $\text{distMm} = \text{echoUs} \times 17 / 100$  (声速 340m/s, 单程距离)。
* distMm : 输出参数, NULL 时仅更新 senserInfo;

```

```

* 成功返回 1，等待超时返回 0。
*/
uint8_t DRIVER_SENSOR_GetHCSR04Distance(uint16_t *distMm){
    uint32_t startCyc, echoStartCyc, echoEndCyc, elapsedUs;

    /* 确保 TRIG 初始为低，等待 2us */
    SENSER_HCSR04_DEP_TRIG_SET(GPIO_LEVEL_LOW);
    SENSER_HCSR04_DEP_DELAY_US(2U);

    /* 发送 10us 触发脉冲 */
    SENSER_HCSR04_DEP_TRIG_SET(GPIO_LEVEL_HIGH);
    SENSER_HCSR04_DEP_DELAY_US(10U);
    SENSER_HCSR04_DEP_TRIG_SET(GPIO_LEVEL_LOW);

    /* 等待 ECHO 上升沿（传感器开始发超声波） */
    startCyc = SENSER_HCSR04_DEP_GET_CYC();
    while(SENSER_HCSR04_DEP_ECHO_READ() == GPIO_LEVEL_LOW){
        if(SENSER_HCSR04_DEP_ELAPSED_US(startCyc, SENSER_HCSR04_START_TIMEOUT_US)){
            return 0U;
        }
    }

    /* 等待 ECHO 下降沿并计时（超声波返回） */
    echoStartCyc = SENSER_HCSR04_DEP_GET_CYC();
    while(SENSER_HCSR04_DEP_ECHO_READ() == GPIO_LEVEL_HIGH){
        if(SENSER_HCSR04_DEP_ELAPSED_US(echoStartCyc, SENSER_HCSR04_ECHO_TIMEOUT_US)){
            return 0U;
        }
    }
    echoEndCyc = SENSER_HCSR04_DEP_GET_CYC();

    /* 计算 ECHO 高电平持续时间（微秒） */
    elapsedUs = (echoEndCyc - echoStartCyc) / dwtInfo.cycPerUs;

    /* 换算距离：单程 = 总时间 / 2，距离(mm) = 时间(us) × 17 / 100 */
    sensorInfo.hcsrDistMm = (uint16_t)(elapsedUs * 17U / 100U);
    sensorInfo.hcsrIsValid = 1U;

    if(distMm != NULL){
        *distMm = sensorInfo.hcsrDistMm;
    }
    return 1U;
}

/* AHT10：触发一次测量，阻塞等待 SENSER_AHT10_MEAS_WAIT_MS ms 后读取 6 字节原始数据，
* 解析温湿度并更新 sensorInfo.aht10TempX100 和 sensorInfo.aht10Humix100。
* 成功返回 1（状态字节忙位为 0），失败返回 0。
*/
uint8_t rxBuf[6];
uint8_t DRIVER_SENSOR_AHT10Update(void){

    uint32_t rawHumi, rawTemp;

```

```

/* 发送触发采集命令 0xAC 0x33 0x00 */
SENDER_AHT10_DEP_WRITE(0xACU, aht10TrigData, 2U);
SENDER_AHT10_DEP_DELAY_MS(SENDER_AHT10_MEAS_WAIT_MS);

/* 读取 6 字节原始数据 */
SENDER_AHT10_DEP_READ(rxBuf, 6U);

/* 检查忙标志位 (bit7=0 表示测量完成) */
if((rxBuf[0] & 0x80U) != 0U){
    return 0U;
}

/* 解析 20 位湿度原始值: rxBuf[1][19:12] rxBuf[2][11:4] rxBuf[3][7:4]→[3:0] */
rawHumi = ((uint32_t)rxBuf[1] << 12U) |
          ((uint32_t)rxBuf[2] << 4U) |
          ((uint32_t)rxBuf[3] >> 4U);

/* 解析 20 位温度原始值: rxBuf[3][3:0]→[19:16] rxBuf[4][15:8] rxBuf[5][7:0] */
rawTemp = (((uint32_t)rxBuf[3] & 0x0FU) << 16U) |
          ((uint32_t)rxBuf[4] << 8U) |
          (uint32_t)rxBuf[5];

/* 换算温度: T(°C) = rawTemp / 2^20 * 200 - 50 */
sensorInfo.aht10Temp = (float)rawTemp / 1048576.0f * 200.0f - 50.0f;

/* 换算湿度: H(%RH) = rawHumi / 2^20 * 100 */
sensorInfo.aht10Humi = (float)rawHumi / 1048576.0f * 100.0f;

sensorInfo.aht10IsValid = 1U;
return 1U;
}

/* AHT10: 返回最近一次测量温度 (单位 °C), 需先调用 DRIVER_SENSEN_AHT10Update. */
float DRIVER_SENSEN_GetAHT10Temp(void){
    return sensorInfo.aht10Temp;
}

/* AHT10: 返回最近一次测量湿度 (单位 %RH), 需先调用 DRIVER_SENSEN_AHT10Update. */
float DRIVER_SENSEN_GetAHT10Humi(void){
    return sensorInfo.aht10Humi;
}

```

task_user1.c:

```

#include "task_user1.h"
#include "driver_ble.h"
#include "driver_board.h"
#include "driver_oled.h"
#include "driver_senser.h"
#include "func_appcom.h"
#include <stdio.h>

```

```

/*=====
 * 私有宏定义
 *=====*/

/* 标定采样次数（每50ms一次，共约500ms完成标定） */
#define CALIB_SAMPLE_COUNT    (10U)
/* 标定超时次数（每50ms加1，共约2000ms；超时后使用默认地面距离） */
#define CALIB_TIMEOUT_COUNT    (40U)
/* 标定超时使用的默认地面距离（mm） */
#define CALIB_DEFAULT_GROUND_MM (500.0f)
/* 液位有效范围下限（mm） */
#define LEVEL_VALID_MIN_MM    (0.0f)
/* 液位有效范围上限（mm） */
#define LEVEL_VALID_MAX_MM    (5000.0f)
/* 滑动均值滤波窗口大小 */
#define FILTER_WIN_SIZE        (5U)

/*=====
 * 公有变量
 *=====*/

/* 液位测量系统任务信息（公有，供 func_appcom 等模块访问） */
user1TaskInfo_t user1TaskInfo;

/*=====
 * 私有函数
 *=====*/

/* 滑动均值滤波：将新值写入循环缓冲区，返回当前窗口均值。
 * newVal ：本次有效液位高度（mm）
 * 返回值 ：滤波后的液位均值（mm）
 */
static float __user1_Filter(float newVal)
{
    float    sum;
    uint8_t count;
    uint8_t i;

    /* 写入新值，推进写入索引 */
    user1TaskInfo.filterBuf[user1TaskInfo.filterIdx] = newVal;
    user1TaskInfo.filterIdx = (uint8_t)((user1TaskInfo.filterIdx + 1U) % FILTER_WIN_SIZE);
    if (user1TaskInfo.filterIdx == 0U) {
        user1TaskInfo.filterFull = 1U; /* 索引回绕，缓冲区已填满 */
    }

    /* 计算有效样本均值 */
    count = user1TaskInfo.filterFull ? FILTER_WIN_SIZE : user1TaskInfo.filterIdx;
    sum = 0.0f;
    for (i = 0U; i < count; i++) {
        sum += user1TaskInfo.filterBuf[i];
    }
    return sum / (float)count;
}

```

```

/* 刷新 OLED 显示内容：
 *   校准中（calibDone==0）：显示系统名称和校准提示；
 *   正常工作（calibDone==1）：显示液位、变化量、距液面距离、高报警阈值。
 *   状态切换时先清屏避免残留字符，同一状态内直接覆盖写入避免闪烁。
 */
static void __user1_OledRefresh(void)
{
    /* 上次刷新时的标定状态，初始化为无效值以保证首次必清屏 */
    static uint8_t prevCalibDone = 0xFFU;
    char buf[32];

    /* 检测状态切换，切换时先清屏 */
    if (prevCalibDone != user1TaskInfo.calibDone) {
        DRIVER_OLED_Clear();
        prevCalibDone = user1TaskInfo.calibDone;
    }

    if (user1TaskInfo.calibDone == 0U) {
        /* ---- 校准中状态显示 ---- */
        DRIVER_OLED_ShowString(0, 0, "LiqLevel System ");
        DRIVER_OLED_ShowString(0, 8, "Calibrating... ");
        DRIVER_OLED_ShowString(0, 16, "Please wait... ");
        DRIVER_OLED_ShowString(0, 24, " ");
    } else {
        /* ---- 正常工作状态显示 ---- */

        /* 第0行：液位高度 */
        sprintf(buf, "Lv1:%5d mm ", (int)user1TaskInfo.liquidLevel_mm);
        DRIVER_OLED_ShowString(0, 0, buf);

        /* 第1行：液位变化量（带正负号） */
        sprintf(buf, "Dlt:%+5d mm ", (int)user1TaskInfo.liquidLevelDelta_mm);
        DRIVER_OLED_ShowString(0, 8, buf);

        /* 第2行：模块距液面原始距离 */
        sprintf(buf, "Dst:%5d mm ", (int)user1TaskInfo.distToLiquid_mm);
        DRIVER_OLED_ShowString(0, 16, buf);

        /* 第3行：高报警阈值（未设置时显示 ----） */
        if (user1TaskInfo.alarmHigh_mm > 0.0f) {
            sprintf(buf, "Alm:%5d mm ", (int)user1TaskInfo.alarmHigh_mm);
        } else {
            sprintf(buf, "Alm: ---- ");
        }
        DRIVER_OLED_ShowString(0, 24, buf);
    }

    DRIVER_OLED_Refresh();
}

/*=====
 * 任务入口

```

```

=====*/

/* 初始化所有驱动和任务内部状态 */
void user1TaskInit(void)
{
    uint8_t i;

    /* 初始化所需驱动 */
    DRIVER_BLE_Init();
    DRIVER_SENSEN_Init();
    DRIVER_OLED_Init();

    /* 初始化内部状态 */
    user1TaskInfo.taskCnt          = 0U;
    user1TaskInfo.calibDone        = 0U;
    user1TaskInfo.calibCnt         = 0U;
    user1TaskInfo.calibTimeoutCnt  = 0U;
    user1TaskInfo.calibSum         = 0.0f;
    user1TaskInfo.distToGround_mm  = 0.0f;
    user1TaskInfo.distToLiquid_mm  = 0.0f;
    user1TaskInfo.liquidLevel_mm   = 0.0f;
    user1TaskInfo.liquidLevelInit_mm = 0.0f;
    user1TaskInfo.liquidLevelDelta_mm = 0.0f;
    user1TaskInfo.alarmHigh_mm     = 200.0f; /* 默认高报警阈值200mm */
    user1TaskInfo.alarmLow_mm      = 0.0f;
    user1TaskInfo.filterIdx        = 0U;
    user1TaskInfo.filterFull       = 0U;
    for (i = 0U; i < FILTER_WIN_SIZE; i++) {
        user1TaskInfo.filterBuf[i] = 0.0f;
    }
}

/* 液位测量系统主任务（2ms基础周期）
 * 调度表：
 *   50ms   (%25==0)：测距采样、标定/液位计算、读取APP报警阈值
 *   100ms  (%50==0)：上报数据至 remoteVar_TX
 *   200ms  (%100==0)：刷新 OLED 显示
 */
void user1TaskUpdata(void *argument)
{
    uint16_t rawDist; /* 超声波测距原始值（mm） */
    float    rawLevel; /* 原始液位高度（mm），未滤波 */
    float    rxHigh;   /* APP 下发的高报警阈值 */
    float    rxLow;    /* APP 下发的低报警阈值 */

    user1TaskInit();
    for (;;) {
        user1TaskInfo.taskCnt++;

        /* ---- 50ms: 测距与计算 ---- */
        if (user1TaskInfo.taskCnt % 25U == 0U) {

            if (user1TaskInfo.calibDone == 0U) {

```

```

/* 功能模块1：上电标定 -- 采集普通超声波地面距离均值 */
user1TaskInfo.calibTimeoutCnt++;

if (DRIVER_SENSEN_GetHCSR04Distance(&rawDist) == 1U) {
    if (rawDist > 0U) {
        user1TaskInfo.calibSum += (float)rawDist;
        user1TaskInfo.calibCnt++;
    }
}

/* 采集满10次正常完成，或超时则使用默认地面距离 */
if ((user1TaskInfo.calibCnt >= CALIB_SAMPLE_COUNT) ||
    (user1TaskInfo.calibTimeoutCnt >= CALIB_TIMEOUT_COUNT)) {

    if (user1TaskInfo.calibCnt >= CALIB_SAMPLE_COUNT) {
        /* 正常完成：使用实测均值 */
        user1TaskInfo.distToGround_mm = user1TaskInfo.calibSum /
(float)CALIB_SAMPLE_COUNT;
    } else {
        /* 超时：使用默认地面距离 500mm */
        user1TaskInfo.distToGround_mm = CALIB_DEFAULT_GROUND_MM;
    }

    /* 读取一次高精度超声波，记录初始液位高度 */
    if (DRIVER_SENSEN_GetDistance(&rawDist) == 1U) {
        user1TaskInfo.distToLiquid_mm = (float)rawDist;
        user1TaskInfo.liquidLevelInit_mm = user1TaskInfo.distToGround_mm -
user1TaskInfo.distToLiquid_mm;
        user1TaskInfo.liquidLevel_mm = user1TaskInfo.liquidLevelInit_mm;
        user1TaskInfo.liquidLevelDelta_mm = 0.0f;
    }
    user1TaskInfo.calibDone = 1U;
}

} else {
    /* 功能模块2：液位高度实时测量与滤波 */
    if (DRIVER_SENSEN_GetDistance(&rawDist) == 1U) {
        rawLevel = user1TaskInfo.distToGround_mm - (float)rawDist;

        /* 有效性校验：超出量程范围则丢弃本次数据 */
        if ((rawLevel >= LEVEL_VALID_MIN_MM) && (rawLevel <= LEVEL_VALID_MAX_MM)) {
            user1TaskInfo.liquidLevel_mm = __user1_Filter(rawLevel);
            user1TaskInfo.distToLiquid_mm = (float)rawDist;
            user1TaskInfo.liquidLevelDelta_mm = user1TaskInfo.liquidLevel_mm -
user1TaskInfo.liquidLevelInit_mm;
        }
    }
}

/* 功能模块3：读取 APP 下发报警阈值 (>0 时认为用户有设置意图) */
rxHigh = remoteInfo.remoteVar_RX[0].var_float;
rxLow = remoteInfo.remoteVar_RX[1].var_float;
if (rxHigh > 0.0f) {
    user1TaskInfo.alarmHigh_mm = rxHigh;
}

```

```

    }
    if (rxLow > 0.0f) {
        user1TaskInfo.alarmLow_mm = rxLow;
    }
}
}

/* ---- 100ms: 上报数据至 APP ---- */
if (user1TaskInfo.taskCnt % 50U == 0U) {
    /* 功能模块4: 仅写本项目使用的索引, 未使用的索引不操作 */
    remoteInfo.remoteVar_TX[4].var_float = user1TaskInfo.liquidLevel_mm;
    remoteInfo.remoteVar_TX[5].var_float = user1TaskInfo.liquidLevelDelta_mm;
    remoteInfo.remoteVar_TX[6].var_uint32 = (uint32_t)user1TaskInfo.calibDone;
    /* [3] 保留, 不操作 */
    remoteInfo.remoteVar_TX[8].var_float = user1TaskInfo.distToLiquid_mm;
    remoteInfo.remoteVar_TX[9].var_float = user1TaskInfo.alarmHigh_mm;
    /* [6][7] 保留, 不操作 */
}

/* ---- 200ms: 刷新 OLED ---- */
if (user1TaskInfo.taskCnt % 100U == 0U) {
    /* 功能模块5: OLED 显示刷新 */
    __user1_OledRefresh();
}

/* ---- 每周期: 报警硬件输出 ---- */
if ((user1TaskInfo.calibDone == 1U) &&
    (user1TaskInfo.alarmHigh_mm > 0.0f) &&
    (user1TaskInfo.liquidLevel_mm >= user1TaskInfo.alarmHigh_mm)) {
    /* 液位超阈值: 点亮红灯 */
    DRIVER_BOARD_RgbOn(BOARD_RGB_R);
    /* 每1000ms触发一次200ms蜂鸣(仅在蜂鸣器空闲时写入, 避免重复触发) */
    if ((user1TaskInfo.taskCnt % 500U == 0U) && (boardInfo.buzzTimeMs == 0U)) {
        boardInfo.buzzTimeMs = 200U;
    }
} else {
    /* 液位正常或标定中: 关闭红灯, 停止蜂鸣 */
    DRIVER_BOARD_RgbOff(BOARD_RGB_R);
    boardInfo.buzzTimeMs = 0U;
}

osDelay(2);
}
}

```

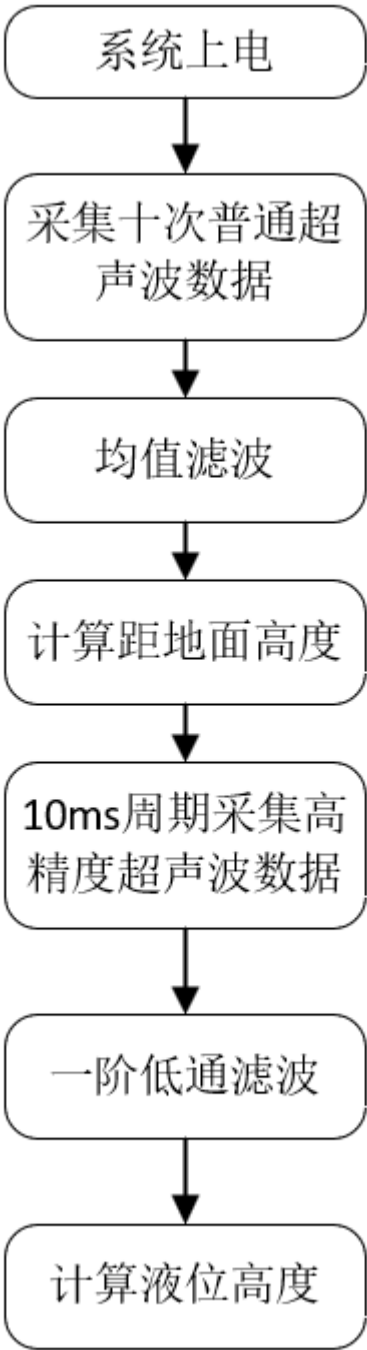
(如有其他应用层文件, 按上述格式追加)

P2.2.2 功能流程图

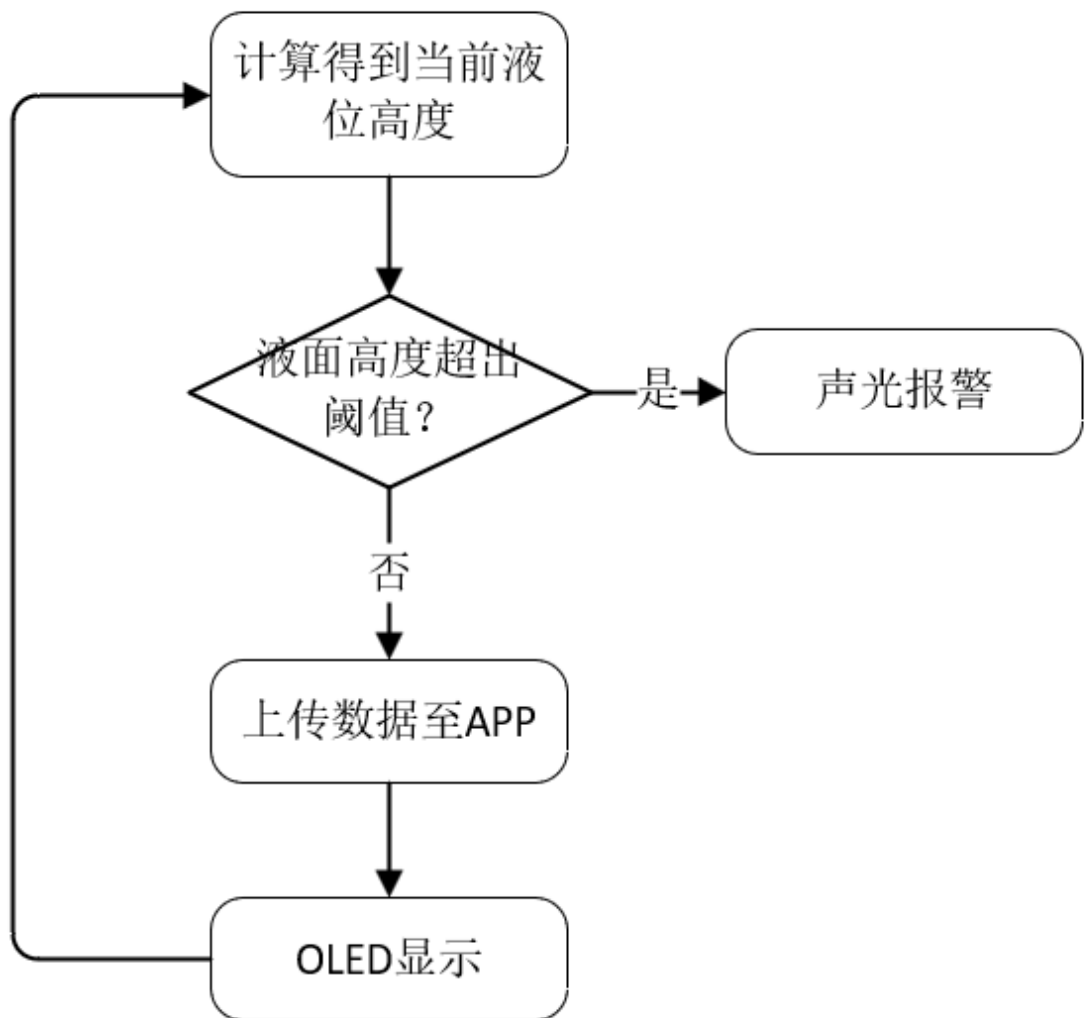
每个功能配一张流程图，Claude 据流程图 + 对应代码段撰写每个"功能 X 软件设计"小节。
小节数量和名称由 Claude 根据代码和流程图自动决定。
图片按 软件流程图_1.png、 软件流程图_2.png 顺序命名。

功能 1:

•



功能 2:



- 描述:

功能 3:

- 描述:

(按需添加)

P2.2.3 上位机软件（可选）

是否包含上位机：是

上位机软件核心代码:

```
package com.example.demo_1

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun HomeScreen(
```

```

        modifier: Modifier = Modifier,
        onAutoConfigureRequested: (ConnectedBleDeviceInfo, List<BleGattServiceUi>) -> Unit = {
            _ , _ -> },
        isPullRefreshing: Boolean = false,
        onRefreshRequested: () -> Unit = {}
    ) {
        val connectedDevice = BleConnectionManager.connectedDeviceInfo
        val discoveredServices = BleConnectionManager.discoveredServices

        if (!UserConfig.DEVELOPER_MODE && connectedDevice != null) {
            LaunchedEffect(
                connectedDevice.address,
                connectedDevice.name,
                discoveredServices,
                UserConfig.auto_config_ble_device,
                UserConfig.esp32_device_name,
                UserConfig.esp32_service_1_uuid,
                UserConfig.esp32_service_1_characteristic_1_uuid,
                UserConfig.esp32_service_2_uuid,
                UserConfig.esp32_service_2_characteristic_1_uuid,
                UserConfig.esp32_service_2_characteristic_2_uuid,
                UserConfig.esp32_service_2_characteristic_3_uuid
            ) {
                onAutoConfigureRequested(connectedDevice, discoveredServices)
            }
        }
    }

    // ===== [修改开始] 液位测量系统 UI =====

    // 读取 BLE 帧数据
    val frame1 = BleProtocol.rxFrames[0x18]
    val frame2 = BleProtocol.rxFrames[0x19]
    val liquidLevel_mm = Float.fromBits(frame1?.var4b1 ?: 0) // 液位高度
    val liquidDelta_mm = Float.fromBits(frame1?.var4b2 ?: 0) // 变化量（带正负）
    val systemStatus = frame1?.var1b1 ?: 0 // 0=校准中，1=正常
    val distToLiquid_mm = Float.fromBits(frame2?.var4b1 ?: 0) // 距液面距离
    val alarmHighEcho_mm = Float.fromBits(frame2?.var4b2 ?: 0) // 高限回显

    // 超限报警状态
    var showAlarmDialog by remember { mutableStateOf(false) }
    var alarmSnoozed by remember { mutableStateOf(false) }
    var alarmCooldown by remember { mutableStateOf(false) }
    val coroutineScope = rememberCoroutineScope()

    val alarmConditionMet = alarmHighEcho_mm > 0f && systemStatus != 0 && liquidLevel_mm >
alarmHighEcho_mm

    // 高限回显变化时重置"不再提醒"
    LaunchedEffect(alarmHighEcho_mm) { alarmSnoozed = false }

    // 满足超限条件且不在冷却/已静音时弹出报警
    LaunchedEffect(alarmConditionMet, alarmCooldown, alarmSnoozed) {
        if (alarmConditionMet && !alarmSnoozed && !alarmCooldown && !showAlarmDialog) {

```

```

        showAlertDialog = true
    }
}

if (showAlertDialog) {
    AlertDialog(
        onDismissRequest = {},
        title = { Text("液位超限报警") },
        text = {
            Text("当前液位 %.0f mm 已超过设定高限 %.0f mm! ".format(liquidLevel_mm,
alarmHighEcho_mm))
        },
        confirmButton = {
            TextButton(onClick = {
                showAlertDialog = false
                alarmCooldown = true
                coroutineScope.launch {
                    delay(10_000L)
                    alarmCooldown = false
                }
            }) { Text("确认") }
        },
        dismissButton = {
            TextButton(onClick = {
                showAlertDialog = false
                alarmSnoozed = true
            }) { Text("不再提醒") }
        }
    )
}

fun sendAlarmThreshold(highMm: Float, lowMm: Float) {
    val frame = BleProtocol.buildTxFrame(
        cmd      = 0x21,
        var4_1 = highMm.toBits(),
        var4_2 = lowMm.toBits(),
        var1_1 = 0,
        var1_2 = 0
    )
    val ok = BleConnectionManager.writeCharacteristic(
        serviceUuid      = UserConfig.esp32_service_1_uuid,
        characteristicUuid = UserConfig.esp32_service_1_characteristic_1_uuid,
        value              = frame
    )
    if (ok) BleConnectionManager.recordOutgoingMessage(
        characteristicUuid = UserConfig.esp32_service_1_characteristic_1_uuid,
        value              = frame
    )
}

val pageContent: @Composable (Modifier) -> Unit = { containerModifier ->
    Column(
        modifier = containerModifier
    )
}

```

```

        .fillMaxSize()
        .verticalScroll(rememberScrollState())
        .padding(horizontal = 16.dp, vertical = 12.dp),
verticalArrangement = Arrangement.spacedBy(16.dp)
) {
    // 顶部标题区域
    Text(
        text = "液位高度测量系统",
        style = MaterialTheme.typography.headlineMedium,
        fontWeight = FontWeight.Bold,
        textAlign = TextAlign.Center,
        modifier = Modifier.fillMaxWidth()
    )
    Text(
        text = "刘皓然",
        style = MaterialTheme.typography.titleLarge,
        color = MaterialTheme.colorsScheme.onSurfaceVariant,
        textAlign = TextAlign.Center,
        modifier = Modifier
            .fillMaxWidth()
            .padding(bottom = 4.dp)
    )

    // BLE 连接状态
    HomeSection(
        title = "BLE 连接状态",
        cardContainerColor = MaterialTheme.colorsScheme.surfaceVariant
    ) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(horizontal = 16.dp, vertical = 12.dp),
            verticalArrangement = Arrangement.spacedBy(6.dp)
        ) {
            if (connectedDevice == null) {
                Text(
                    text = "未连接",
                    style = MaterialTheme.typography.titleMedium,
                    color = MaterialTheme.colorsScheme.onSurfaceVariant
                )
                Text(
                    text = "下拉刷新可重新搜索设备",
                    style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorsScheme.onSurfaceVariant
                )
            } else {
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween
                ) {
                    Text(
                        text = "已连接",
                        style = MaterialTheme.typography.titleMedium,

```

```

        color = MaterialTheme.colorsScheme.primary
    )
    Text(
        text = "${connectedDevice.rssi} dBm",
        style = MaterialTheme.typography.bodyMedium,
        color = MaterialTheme.colorsScheme.onSurface
    )
}
Text(
    text = "设备: ${connectedDevice.name}",
    style = MaterialTheme.typography.bodyMedium,
    color = MaterialTheme.colorsScheme.onSurface
)
Text(
    text = "MAC: ${connectedDevice.address}",
    style = MaterialTheme.typography.bodySmall,
    color = MaterialTheme.colorsScheme.onSurfaceVariant
)
}
}
}

if (connectedDevice != null) {
    // 区域一: 系统状态
    HomeSection(title = "系统状态") {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(horizontal = 16.dp, vertical = 12.dp)
        ) {
            val statusText = if (systemStatus == 0) "正在校准..." else "正常工作"
            val statusColor = if (systemStatus == 0) Color(0xFFFF9800) else
Color(0xFF4CAF50)

            Text(
                text = statusText,
                color = statusColor,
                style = MaterialTheme.typography.titleMedium
            )
        }
    }

    // 区域二: 实时测量数据
    HomeSection(title = "实时测量数据") {
        val isCalib = systemStatus == 0
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(horizontal = 16.dp, vertical = 12.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            ProtoFieldRow(
                label1 = "液位高度",

```

```

        value1 = if (isCalib) "---" else "%.0f
mm".format(liquidLevel_mm),
        label2 = "液位变化量",
        value2 = if (isCalib) "---" else "%+.0f
mm".format(liquidDelta_mm),
        value1Color = if (!isCalib && alarmConditionMet) color.Red else
null
    )
    ProtoFieldRow(
        label1 = "距液面距离",
        value1 = if (isCalib) "---" else "%.0f
mm".format(distToLiquid_mm),
        label2 = "当前高限",
        value2 = if (isCalib) "---"
            else if (alarmHighEcho_mm > 0f) "%.0f
mm".format(alarmHighEcho_mm)
            else "未设置"
    )
}

// 区域三: 报警阈值设置
HomeSection(title = "报警阈值设置") {
    var highInput by remember { mutableStateOf("") }
    var lowInput by remember { mutableStateOf("") }
    val context = LocalContext.current

    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 12.dp)
    ) {
        OutlinedTextField(
            value = highInput,
            onValueChange = { highInput = it },
            label = { Text("高报警阈值 (mm)") },
            placeholder = { Text("如: 800") },
            keyboardOptions = KeyboardOptions(keyboardType =
KeyboardType.Number),
            modifier = Modifier.fillMaxWidth()
        )
        Spacer(modifier = Modifier.height(8.dp))
        OutlinedTextField(
            value = lowInput,
            onValueChange = { lowInput = it },
            label = { Text("低报警阈值 (mm)") },
            placeholder = { Text("如: 200") },
            keyboardOptions = KeyboardOptions(keyboardType =
KeyboardType.Number),
            modifier = Modifier.fillMaxWidth()
        )
        Spacer(modifier = Modifier.height(12.dp))
        Button(

```

```

        onClick = {
            val high = highInput.toFloatOrNull()
            val low = lowInput.toFloatOrNull()
            when {
                high == null || low == null -> Toast.makeText(context,
"请输入有效数值", Toast.LENGTH_SHORT).show()
                high <= 0f || low <= 0f -> Toast.makeText(context,
"阈值须大于0", Toast.LENGTH_SHORT).show()
                high > 5000f || low > 5000f -> Toast.makeText(context,
"阈值超出范围（最大5000mm）", Toast.LENGTH_SHORT).show()
                high <= low -> Toast.makeText(context,
"高限须大于低限", Toast.LENGTH_SHORT).show()
                else -> sendAlarmThreshold(high,
low)
            }
        },
        modifier = Modifier.fillMaxWidth()
    ) {
        Text("设置阈值")
    }
}

}

}

}

}

// ===== [修改结束] =====

if (UserConfig.DEVELOPER_MODE) {
    pageContent(modifier)
} else if (connectedDevice == null) {
    PullToRefreshBox(
        isRefreshing = isPullRefreshing,
        onRefresh = onRefreshRequested,
        modifier = modifier.fillMaxSize()
    ) {
        pageContent(modifier)
    }
} else {
    pageContent(modifier)
}
}

@Composable
private fun HomeSection(
    title: String,
    cardContainerColor: Color = MaterialTheme.colorScheme.surface,
    content: @Composable () -> Unit
) {
    Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
        Text(
            text = title,

```

```
        style = MaterialTheme.typography.bodyMedium,
        color = MaterialTheme.colorsScheme.onSurfaceVariant,
        modifier = Modifier.padding(horizontal = 4.dp)
    )
    Card(
        modifier = Modifier.fillMaxWidth(),
        shape = RoundedCornerShape(22.dp),
        colors = CardDefaults.cardColors(
            containerColor = cardContainerColor
        )
    ) {
        content()
    }
}
}
```

如包含：

上位机UI

GUET

液位高度测量系统

刘浩然

BLE 连接状态

已连接

-87 dBm

设备: Unnamed device

MAC: 44:FF:48:D4:F5:A3

系统状态

正在校准...

实时测量数据

液位高度

液位变化量

距液面距离

当前高限

报警阈值设置

高报警阈值 (mm)

低报警阈值 (mm)

设置阈值



- 描述:

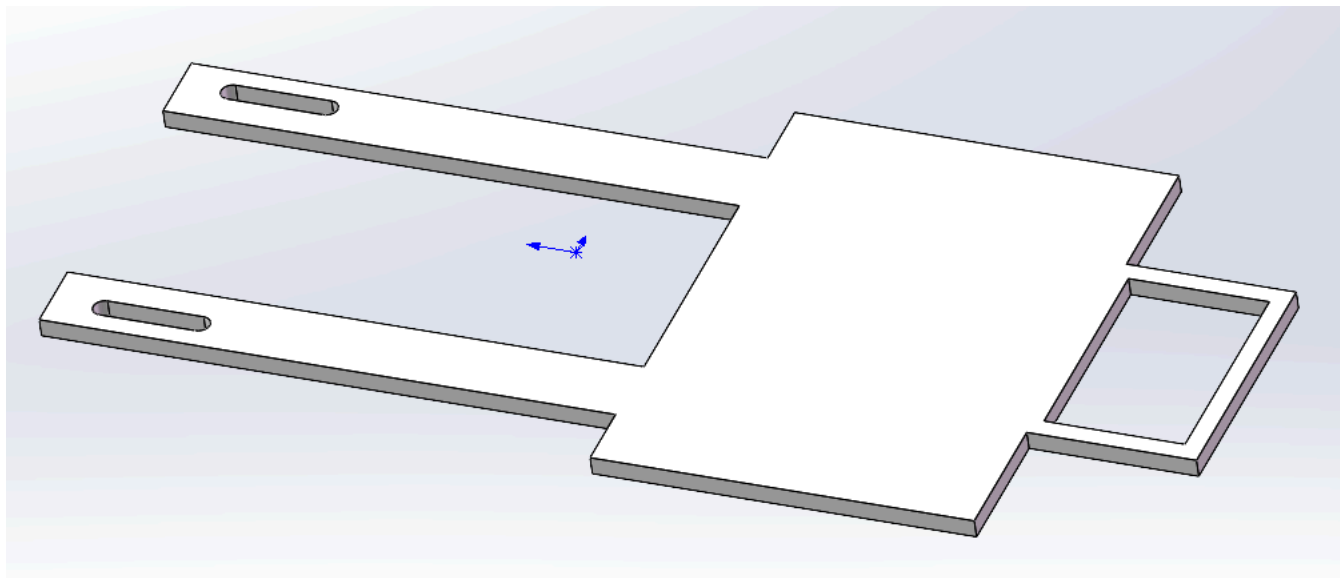
P2.3 机械结构素材（用于第五章，无则填"否"）

是否包含机械结构：是

如包含：

图片按 机械_1.png、机械_2.png 顺序命名。每张图必须出现在论文中。

零件 1：



- 描述：前端用于固定高精度超声波模块，后端用于固定普通超声波模块。中间的平台用于放置PCB板子

零件 2：



- 描述：系统实物图

(按需添加更多零件)

Part 3: 系统测试 + 总结与展望

触发指令: 输出 Part 3

输出章节：第六章系统测试 / 第七章总结与展望

预计字数：3000~4000 字

依赖: G1~G6 + 本 Part 输入

第六章特殊要求：测试小节不能只是“看图说话”，必须结合项目背景做进一步解释：

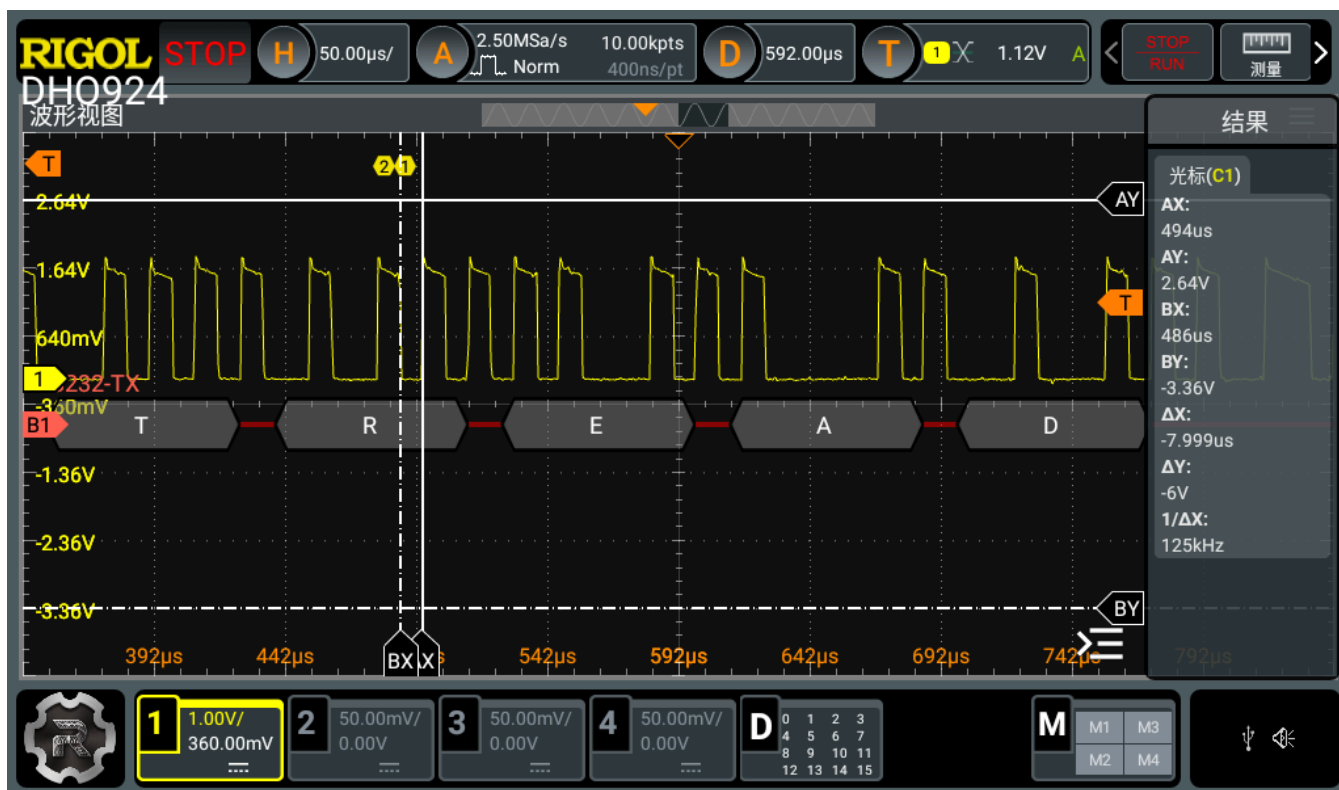
- 把测试现象与第一章的设计目标对应（达没达到指标）
- 把测试结果与第三、四章的硬件/软件设计原理联系起来（为什么会有这个结果）
- 联系实际应用场景说明测试的工程意义
- 对异常或非理想的测试结果做合理分析

P3.1 测试图集（用于第六章，每个功能一组）

每个功能一张测试图 + 简短描述。Claude 据此撰写测试方法、过程、结果、结论，并结合项目背景深入分析。

每张图必须出现在论文中。

测试 1:

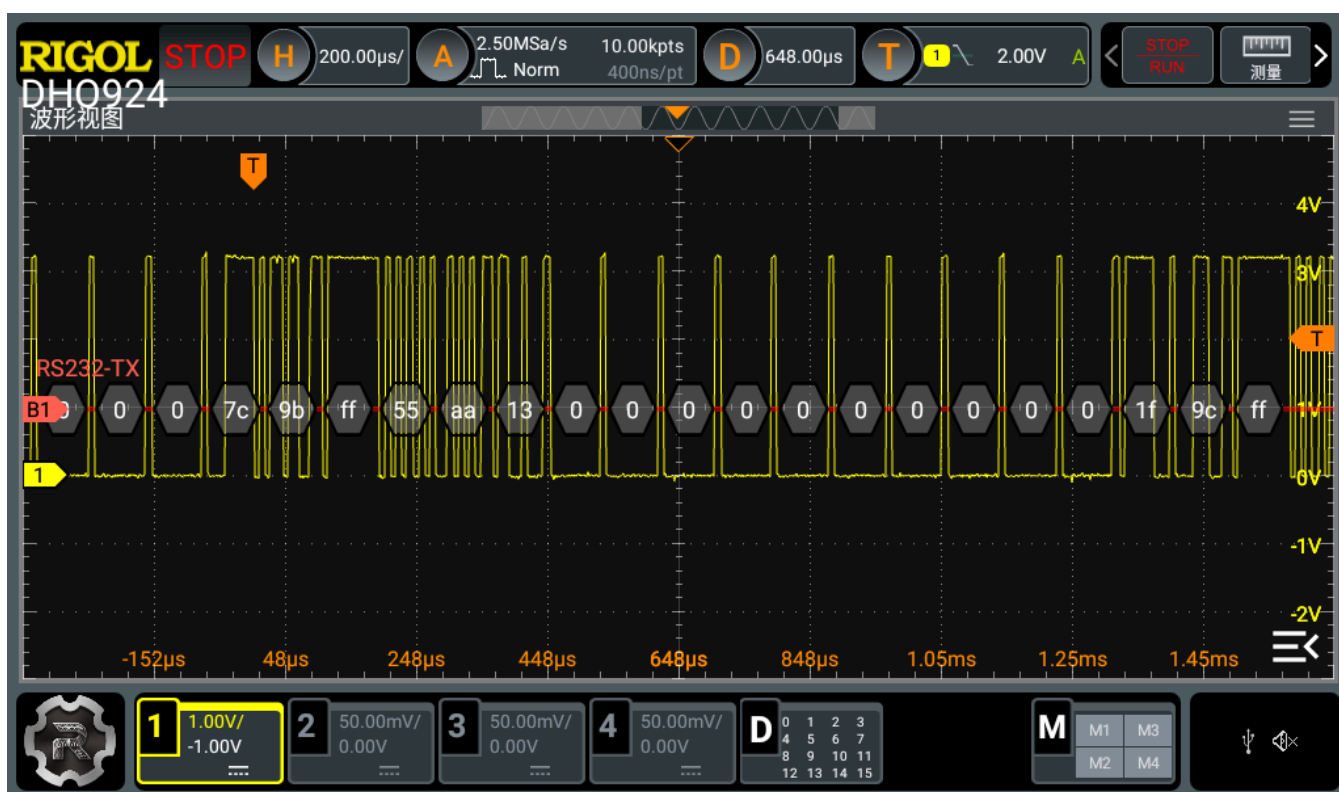


- 描述：超声波模块返回串口数据包，这张图是错的，强行简单解释一下

测试 2:



测试 3:



测试 4:

生成30组液面高度测量数据，满足题目指标要求。生成表格数据并进行描述。关于探测距离，相应时间，误差，分辨率等指标进行多次（至少30次）测量计算均值和方差

(按需添加)

P3.2 不足与改进（可选）

诚实列 2~4 条本设计的不足。这部分老师很看重。
不填则 Claude 按项目特点自动生成 3 条通用不足。

- 1.
- 2.
- 3.

【留空则 Claude 自由发挥】

使用说明

第 1 轮：

- 填写 G1~G6 + P1.1 + P1.2 + P1.3
- 指令：输出 Part 1
- → 摘要 + 绪论 + 系统方案设计

第 2 轮：

- 填写 P2.1~P2.3
- 指令：输出 Part 2
- → 硬件设计 + 软件设计 + 机械外观设计

第 3 轮：

- 填写 P3.1~P3.2
- 指令：输出 Part 3
- → 系统测试 + 总结与展望

完成后可指令：整合完整论文 → Claude 输出最终单文件 md

—— 以下为项目软件实现的内部知识（Claude 自动参考，不要修改） ——

重要：以下内容仅供 Claude 撰写论文时理解项目软件实现的内部细节，使第四章软件设计写得专业、贴合实际代码。

⚠ 论文中绝对不允许出现以下表述：

- 不得提到"框架"、"软件框架"、"已有框架"、"项目模板"、"代码模板"等任何暗示这是基于已有框架开发的词
- 不得提到"task 层"、"driver 层"、"stdlib 层"、"func 层"、"HAL 层"等分层术语（可改写为"应用层"、"驱动层"、"基础库"、"功能模块"、"硬件抽象层"等通用工程术语）
- 不得提到 `userLib`、`userDriver`、`userFunc`、`userApp` 等具体目录名（可改写为"驱动文件目录"、"应用文件目录"等通用描述）
- 论文作者视角：所有软件结构、命名规范、调度方式都是**作者自己设计的**，不是套用的

改写示例：

- ✗ "本项目采用 task/driver/stdlib 分层框架" → ✓ "本设计采用分层式软件结构，将代码组织为应用层、驱动层和基础库三层"
- ✗ "在 task_user1 中调用 driver 层 API" → ✓ "在主应用任务中调用底层驱动接口"
- ✗ "func_appcom 模块负责通信" → ✓ "通信处理模块负责蓝牙数据收发"

1. 项目软硬件平台

- 主控芯片：STM32F103C8T6（Cortex-M3，72MHz，64KB Flash，20KB SRAM）
- 软件库：STM32 HAL 库
- 操作系统：FreeRTOS（CMSIS_V2 接口）
- 开发环境：MDK-ARM Keil 5
- 配置工具：STM32CubeMX
- 电源方案：7.4V 锂电池 → 两路 LDO 稳压输出 5V + 一路 DCDC 升压 12V（若有）

2. 软件分层结构（论文中需用通用术语改写）

项目软件按职责划分为五层（自上而下）：

内部叫法	论文中表述（建议）	职责
task 层（ <code>userApp</code> ）	应用层 / 任务层	业务逻辑，开发者主要工作区
func 层（ <code>userFunc</code> ）	功能模块层 / 中间件层	跨驱动的逻辑整合，如通信处理
driver 层（ <code>userDriver</code> ）	驱动层	各外设的驱动封装
stdlib 层（ <code>userLib</code> ）	基础库 / 工具库	通用工具函数（延时、CRC、字符串等）
HAL 层（ <code>Core/</code> ）	硬件抽象层	ST 官方 HAL 库

调用规则：上层可调用下层，禁止跨层（应用层不直接调 HAL）。

3. FreeRTOS 任务规划

项目固定使用两个 FreeRTOS 任务：

任务名	周期	职责
task_system	10ms	系统级周期任务，维护基础库状态机和通信更新
task_user1	2ms	应用级任务，承担主要业务逻辑

task_user1 内部通过 taskCnt 计数器取余实现多周期：

```
if (taskCnt % 5 == 0) { /* 10ms 周期逻辑 */ }
if (taskCnt % 50 == 0) { /* 100ms 周期逻辑 */ }
if (taskCnt % 250 == 0) { /* 500ms 周期逻辑 */ }
if (taskCnt % 500 == 0) { /* 1s 周期逻辑 */ }
```

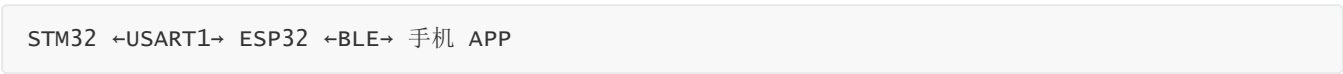
论文中描述：可写为"主任务以 2ms 为基础调度周期，通过计数器取余实现 10ms / 100ms / 500ms / 1s 等多级时间片"。

4. 命名规范（论文中提到代码命名时遵守）

- 任务函数：xxxTaskInit() / xxxTaskUpdata()（注意是 Updata 不是 Update，这是项目代码实际拼写）
- 任务结构体：xxxTaskInfo_t + extern xxxTaskInfo_t xxxTaskInfo
- 驱动函数：DRIVER_XXX_Init() / DRIVER_XXX_Update() / DRIVER_XXX_DoSomething()
- 驱动结构体：xxxInfo 或 xxxInfo[CH_COUNT]
- 命名风格：小驼峰（变量/函数）+ 大驼峰t（结构体）+ 全大写下划线（宏）
- 注释语言：中文
- 内存分配：全静态分配，禁用 malloc

5. 通信链路（若项目有 BLE/APP 通信）

通信链路结构：



- STM32 通过 USART1 与 ESP32 通信，统一使用 16 字节定长帧
- ESP32 作为 BLE 协处理器，将 STM32 数据透传给 APP，反之亦然
- 应用层通过 remoteVar_TX[16] / remoteVar_RX[16] 两个变量池读写，不需要操作底层帧

remoteVar_t 是 4 字节联合体：

```
typedef union {
    uint8_t raw[4];
    uint32_t var_uint32;
    int32_t var_int32;
    float var_float;
} remoteVar_t;
```

槽位有效宽度：

- TX/RX 索引 [0][1][4][5][8][9][12][13] → 4 字节槽位 (float / uint32 / int32)
- TX/RX 索引 [2][3][6][7][10][11][14][15] → 1 字节槽位

关键约束：TX[0]~[3] 不会到达 APP（被 ESP32 截留），发送给 APP 的数据使用 TX[4]~[15]。

论文中描述：可写为"主控通过串口与无线通信模块交换数据，无线通信模块再通过低功耗蓝牙将数据透传至手机端。为简化业务层代码，作者设计了变量池机制，应用层只需读写约定的变量数组即可完成双向通信"。

6. 字节序处理（论文若涉及通信细节）

APP 端以大端序发送 4 字节字段，STM32 端 USART 接收处理函数已自动翻转为小端序。**应用层直接读写 remoteVar 即可，无需关心字节序。**

7. 上位机（若项目有 APP）

- 平台：Android 原生
- UI 框架：Jetpack Compose + Material Design 3
- 通信：BLE GATT
- 主页面：单页 HomeScreen，包含项目名称/作者标题区 + 多个分区
- UI 组件：分区用 HomeSection，数据行用 ProtoFieldRow（标签：值），控制用 Button
- 设计原则：
 - STM32 → APP 的数据 → 文本显示
 - APP → STM32 的控制 → 按钮（不用 Slider/SeekBar）
 - 数值类控制用多个按钮代替（如 [音量+] [音量-] 或 [亮度20%] [亮度50%]）

论文中描述：可写为"上位机界面采用现代化的声明式 UI 框架开发，将界面划分为若干功能分区，数据显示采用文本形式，控制操作统一使用按钮实现，避免滑动控件可能带来的误触和精度问题"。

8. 论文写作指引

撰写第四章软件设计时，Claude 应当：

1. **从作者视角讲述设计思路**，不要写成"项目使用了 XX 框架"——而是"作者经过分析后采用了 XX 设计方案"
2. **代码命名直接使用项目实际命名**（user1TaskInfo、DRIVER_XXX_Init 等），但解释命名时说"作者按照模块化原则采用统一的命名风格"
3. **分层结构用通用工程术语描述**，不要出现 task/driver/stdlib 这种内部叫法
4. **2ms 基础周期 + taskCnt 取余**这套机制要解释为"作者的设计巧思"，而不是"框架要求"
5. **遇到代码细节时**，结合代码本身解释作者的实现思路，让论文显得是真的写过这份代码
6. **每段贴出的代码都要有对应小节解释**，不能贴而不讲